

Competitive Programming

Section 40: Data Structures Mixed Contest

Competitive Programming Course

2026-07-06

Table of contents

1	Combining data-structure ideas	3
2	First-fit with a segment tree	4
3	Removing by order	5
4	Mo's algorithm	6
5	Choosing between similar tools	7
6	Mixed contest solving order	8
7	Contest checklist	9
8	Local exercises	10
9	How to use the checker	11

1 Combining data-structure ideas

Contest data-structure problems rarely say “use a segment tree” or “use a Fenwick tree.” They describe operations. Your job is to map operations to the smallest structure that supports them.

Read operations as contracts:

```
point update + prefix/range sum      -> Fenwick
point update + arbitrary range min    -> segment tree
range update + range query            -> lazy segment tree or two Fenwicks
static range idempotent query         -> sparse table
offline add/remove endpoints          -> Mo's algorithm
find first position satisfying query  -> segment tree descent or Fenwick search
```

2 First-fit with a segment tree

If every hotel has remaining capacity and each group wants the first hotel with capacity at least x , store maximum capacity in each segment tree node. If the root maximum is smaller than x , no hotel works. Otherwise descend:

```
if left_child.max >= x: go left
else: go right
```

This is a binary search guided by aggregate information.

Example:

```
capacities = [2, 8, 4, 6]
group = 5
```

The root maximum is 8, so some hotel works. Check the left half [1,2]:

```
max([2,8]) = 8 >= 5
```

Go left. Then check hotel 1:

```
2 < 5
```

So go to hotel 2, subtract the group, and update capacity $8 \rightarrow 3$.

3 Removing by order

If elements are removed from a list and queries ask for the k-th alive element, store 1 for alive positions and 0 for removed positions. A Fenwick tree can find the smallest index with prefix sum at least k by binary lifting on the Fenwick structure.

Example:

```
alive = [1, 1, 0, 1, 1]
prefix sums = [1, 2, 2, 3, 4]
```

The 3-rd alive element is the first position with prefix sum at least 3, which is position 4.

After removing it:

```
alive = [1, 1, 0, 0, 1]
```

4 Mo's algorithm

Mo's algorithm answers offline range queries by ordering them so the current interval changes slowly. Maintain a window $[L,R]$ and four operations:

```
add_left, add_right, remove_left, remove_right
```

For distinct count, a frequency table plus a **distinct** counter is enough. Mo is useful when updates are absent and adding/removing one endpoint is cheap.

Mo's ordering groups queries by block of L , then sorts by R inside each block:

```
block = L // sqrt(n)  
sort by (block, R)
```

This keeps the window from jumping too far on average. The algorithm is not online: you must know all queries before answering them.

5 Choosing between similar tools

Fenwick and segment trees overlap, but the decision is usually simple:

```
Need prefix sums or order statistics? Fenwick is shorter.  
Need min/max over arbitrary intervals? Segment tree.  
Need first index whose segment aggregate satisfies a condition? Segment tree.  
Need k-th alive by prefix counts? Fenwick binary lifting.
```

Mo's algorithm is different: it is not a tree. It trades online answers for an offline ordering that makes add/remove operations cheap. Use it when the query function is easy to update after moving one endpoint but hard to merge from two children.

6 Mixed contest solving order

When a section mixes structures, do not pick the first familiar tool. Write the operation table:

```
update type:  
query type:  
online/offline:  
need first position or only value:  
can answers be reordered:
```

For Hotel Queries, “first position” is the clue. For List Removals, “k-th alive” is the clue. For Mo Distinct Queries, “offline range queries with cheap add/remove” is the clue.

7 Contest checklist

Are operations online or can queries be reordered?

Is the query about prefix/rank/order? Consider Fenwick.

Is the query about arbitrary intervals? Consider segment tree.

Is the array static and queries many? Consider sparse table or Mo.

8 Local exercises

- A. Hotel Queries: For each group size, place it in the first hotel with enough remaining capacity, subtract that size, and print the hotel index or 0.
- B. List Removals: Repeatedly remove and print the k -th currently alive element.
- C. Mo Distinct Queries: For each offline range query, print the number of distinct values in the range.

9 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/40_data_structures_mixed_contest/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.